
SmolVLA + Koch v1.1

소스코드 가이드

src/ 폴더 Python 코드 상세 해설

RISE 창업동아리

국립목포해양대학교

2026년 4월

문서 개요

src/ 폴더에 포함된 8개 Python 스크립트의 구조, 동작 원리, 주요 코드를 상세히 설명한다.

Part A (01-04, 08): 로봇 없이 PC만으로 실행 가능

Part B (05, 07): Koch v1.1 로봇팔 필요

06: 수집된 데이터셋이 Hub에 있으면 로봇 없이 실행 가능

Contents

1	파일 개요	2
2	01_test_install.py — 설치 확인	3
2.1	목적	3
2.2	동작 흐름	3
2.3	주요 코드	3
3	02_explore_dataset.py — 공개 데이터셋 탐색	4
3.1	목적	4
3.2	동작 흐름	4
3.3	주요 코드 — 데이터셋 로드 및 프레임 구조 확인	4
3.4	주요 코드 — Action 통계	4
4	03_train_sim_diffusion.py — 시뮬레이션 학습	5
4.1	목적	5
4.2	동작 흐름	5
4.3	핵심 개념 — delta_timestamps	5
4.4	핵심 개념 — 학습 루프	5
5	04_inference_no_robot.py — 추론 테스트	7
5.1	목적	7
5.2	동작 흐름	7
5.3	주요 코드 — 추론 루프	7
6	05_collect_data.py — 시연 데이터 수집	8
6.1	목적	8
6.2	동작 흐름	8
6.3	주요 코드 — 로봇 설정	8
6.4	주요 코드 — 녹화 루프	8
7	06_finetune_smolvla.py — SmolVLA Fine-tuning	10
7.1	목적	10
7.2	동작 흐름	10
7.3	주요 코드 — 학습 루프	10
8	07_run_policy_koch.py — 자율 제어	11
8.1	목적	11
8.2	동작 흐름	11
8.3	주요 코드 — 자율 제어 실행	11
9	08_sim_smolvla_koch.py — SmolVLA + Koch 시뮬레이션	13
9.1	목적	13
9.2	모드 요약	13
9.3	내부 구성 요소	13
9.3.1	(1) Koch 시뮬레이션 엔진	13
9.3.2	(2) KochSimEnv — 시뮬레이션 환경 클래스	13
9.3.3	(3) SmolVLASimWrapper — SmolVLA 래퍼	14
9.4	SmolVLA 추론 파이프라인 상세	14
9.5	모드별 실행	14

10 09_voice_pick_place.py — 음성 Pick & Place 시뮬레이션	16
10.1 목적	16
10.2 모드 요약	16
10.3 시뮬레이션 씬 구성	16
10.4 동작 흐름	16
10.5 내부 구성 요소	17
10.5.1 (1) PickPlaceSimulator — 시뮬레이션 + 물리 엔진	17
10.5.2 (2) CommandParser — 자연어 명령 파서	17
10.5.3 (3) VoiceInput — 음성 인식	17
10.5.4 (4) Multi-Start CCD 역기구학	18
10.6 Pick & Place 시퀀스 상세	18
10.7 실행 방법	18
11 스크립트 간 관계 요약	20

1 파일 개요

Table 1: 소스코드 파일 목록 및 역할

파일	로봇	GPU	역할	핵심 기능
01_test_install.py	-	-	설치 확인	패키지 버전, GPU, SmolVLA импорт 확인
02_explore_dataset.py	-	-	데이터 탐색	PushT 데이터셋 구조 파악
03_train_sim_diffusion.py	-	권장	시물 학습	Diffusion Policy 학습 루프
04_inference_no_robot.py	-	권장	추론 테스트	사전학습 모델 예측 vs GT 비교
05_collect_data.py	필요	-	데이터 수집	텔레오퍼레이션 시연 녹화
06_finetune_smolvla.py	-	필요	Fine-tuning	SmolVLA 모델 학습
07_run_policy_koch.py	필요	권장	자율 제어	학습 모델로 로봇 실행
08_sim_smolvla_koch.py	-	권장	시물+VLA	SmolVLA로 시물 Koch 제어
09_voice_pick_place.py	-	-	음성 P&P	음성/텍스트 명령으로 Pick & Place

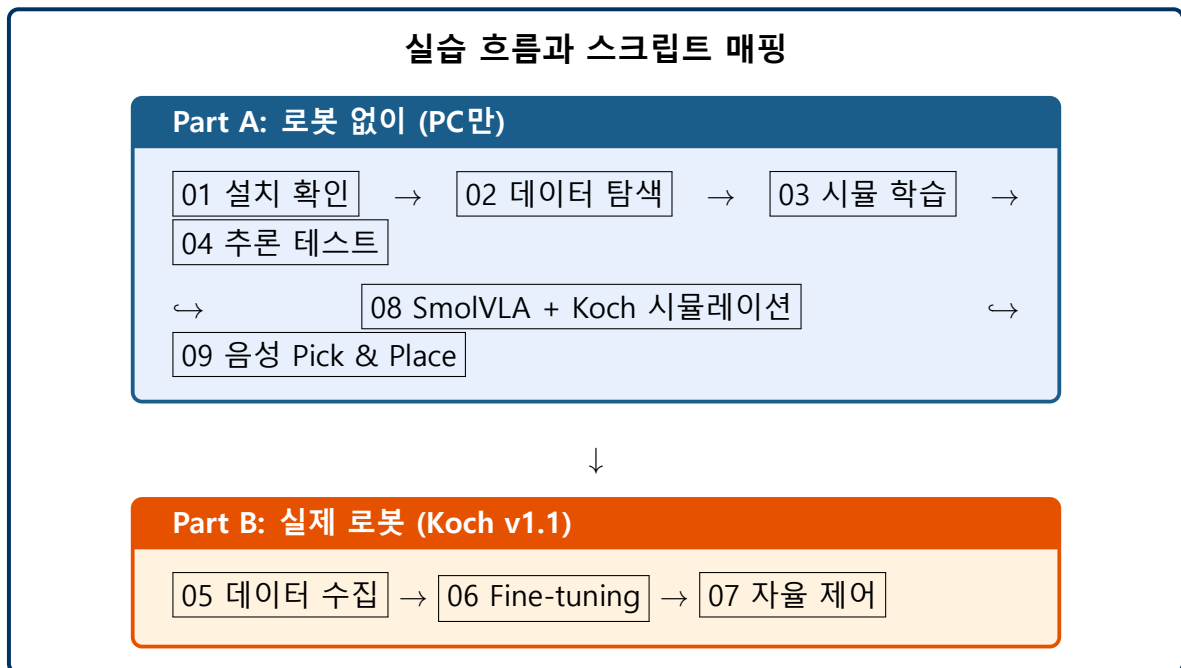


Figure 1: 스크립트 실행 순서

2 01_test_install.py — 설치 확인

```
src/01_test_install.py
```

2.1 목적

LeRobot, PyTorch, CUDA, SmolVLAPolicy가 정상적으로 설치되었는지 한 번에 확인한다. 환경 설정 후 **가장 먼저** 실행해야 하는 스크립트이다.

2.2 동작 흐름

1. lerobot.__version__, torch.__version__ 출력
2. torch.cuda.is_available()로 GPU 사용 가능 여부 확인
3. GPU가 있으면 이름과 VRAM 용량 출력, Apple Silicon이면 MPS 표시
4. SmolVLAPolicy 클래스를 import하여 성공/실패 출력

2.3 주요 코드

```
1 import torch
2 import lerobot
3
4 print(f"LeRobot version : {lerobot.__version__}")
5 print(f"CUDA available : {torch.cuda.is_available()}")
6
7 if torch.cuda.is_available():
8     print(f"GPU : {torch.cuda.get_device_name(0)}")
9     mem = torch.cuda.get_device_properties(0).total_memory
10    print(f"VRAM : {mem / 1e9:.1f} GB")
11
12 from lerobot.policies.smolvla.modeling_smolvla import SmolVLAPolicy
13 print("SmolVLAPolicy : import OK")
```

확인 포인트

- CUDA available: True이면 GPU 학습/추론 가능
- SmolVLAPolicy: import OK이면 SmolVLA 관련 의존성 정상
- 실패 시 pip install "lerobot[smolvla]" 재설치

실행 방법

```
python src/01_test_install.py
```

3 02_explore_dataset.py — 공개 데이터셋 탐색

```
src/02_explore_dataset.py
```

3.1 목적

HuggingFace Hub의 lerobot/pusht 데이터셋을 다운로드하여 구조(키, shape, dtype)를 파악하고, action 분포를 통계적으로 확인한다. 이미지 관측이 있으면 시각화하여 PNG로 저장한다.

3.2 동작 흐름

1. LeRobotDataset("lerobot/pusht", episodes=[0,1,2])로 에피소드 3개 로드
2. 총 프레임 수, FPS 등 메타데이터 출력
3. dataset[0]에서 프레임 구조(키 목록, 텐서 shape/dtype) 출력
4. 전체 프레임의 action 텐서를 모아 mean, std, min, max 통계 계산
5. observation.image가 있으면 프레임 0-4를 matplotlib으로 시각화 후 pusht_frames_sample.png로 저장

3.3 주요 코드 — 데이터셋 로드 및 프레임 구조 확인

```
1 from lerobot.datasets.lerobot_dataset import LeRobotDataset
2
3 dataset = LeRobotDataset("lerobot/pusht", episodes=[0, 1, 2])
4 frame = dataset[0]
5
6 for key, value in frame.items():
7     if isinstance(value, torch.Tensor):
8         print(f" {key:30s} | shape={value.shape} | dtype={value.dtype}")
```

3.4 주요 코드 — Action 통계

```
1 actions = torch.stack([dataset[i]["action"] for i in range(dataset.num_frames)])
2 print(f"Action mean : {actions.mean(dim=0).numpy()}")
3 print(f"Action std  : {actions.std(dim=0).numpy()}")
```

PushT 데이터셋 구조

- observation.image: (3, 96, 96) float32 — RGB 이미지
- observation.state: (2,) float32 — 에이전트 위치 (x, y)
- action: (2,) float32 — 이동 방향 (dx, dy)
- FPS: 10, 에피소드당 약 125 프레임

실행 방법

```
python src/02_explore_dataset.py
```

4 03_train_sim_diffusion.py — 시뮬레이션 학습

src/03_train_sim_diffusion.py

4.1 목적

PushT 환경에서 **Diffusion Policy**를 학습한다. Hub의 사전학습 모델에서 config를 가져와 동일한 구조로 처음부터 학습하는 전체 파이프라인을 보여준다.

4.2 동작 흐름

1. DiffusionPolicy.from_pretrained("lerobot/diffusion_pusht")로 config 확인
2. delta_timestamps를 설정하여 temporal window가 포함된 데이터셋 로드
3. DataLoader 생성 (num_workers=0: Windows 호환)
4. 모델을 GPU로 이동, Adam 옵티마이저 설정
5. 학습 루프: policy.forward(batch) → loss → backward → step
6. 학습 완료 후 outputs/train/diffusion_pusht/에 체크포인트 저장

4.3 핵심 개념 — delta_timestamps

Diffusion Policy는 시간적 맥락(temporal context)이 필요하다. delta_timestamps는 현재 프레임 기준으로 어떤 시점의 데이터를 함께 로드할지 지정한다.

```

1 fps = 10
2 dt = 1.0 / fps
3 # n_obs_steps=2: 현재 + 이전 프레임 관측
4 # horizon=16: 16 스텝 행동 시퀀스
5 delta_timestamps = {
6     "observation.image": [dt*(i - cfg.n_obs_steps+1) for i in range(cfg.
7         n_obs_steps)],
8     "observation.state": [dt*(i - cfg.n_obs_steps+1) for i in range(cfg.
9         n_obs_steps)],
10    "action": [dt*(i - cfg.n_obs_steps+1) for i in range(cfg.horizon)],
11 }

```

4.4 핵심 개념 — 학습 루프

```

1 for batch in dataloader:
2     batch_gpu = {k: v.to(device) if isinstance(v, torch.Tensor) else v
3         for k, v in batch.items()}
4
5     loss, _ = policy.forward(batch_gpu)
6
7     optimizer.zero_grad()
8     loss.backward()
9     optimizer.step()

```

설정값 참고

- NUM_STEPS=300: 데모용 빠른 테스트. 실제 학습은 **100,000+** 스텝 필요
- BATCH_SIZE=8: 8GB VRAM 기준. 부족하면 4 또는 2로 줄인다
- 체크포인트: outputs/train/diffusion_pusht/ 에 저장

실행 방법

```
python src/03_train_sim_diffusion.py
```


5 04_inference_no_robot.py — 추론 테스트

```
src/04_inference_no_robot.py
```

5.1 목적

사전학습된 Diffusion Policy를 로드하여 PushT 데이터셋의 프레임에 대해 행동을 **예측**하고, 실제 행동(ground truth)과 비교한다. 로봇 없이 모델의 동작 원리를 확인할 수 있다.

5.2 동작 흐름

1. DiffusionPolicy.from_pretrained("lerobot/diffusion_pusht") 로드
2. policy.eval() 모드 설정 후 policy.reset() 호출
3. 각 프레임에 대해:
 - (a) 배치 차원 추가 (unsqueeze(0)) 후 GPU 이동
 - (b) policy.select_action(batch)로 행동 예측
 - (c) 예측값과 GT를 테이블 형태로 출력

5.3 주요 코드 — 추론 루프

```

1 policy.reset()
2
3 with torch.no_grad():
4     for i in range(NUM_FRAMES):
5         frame = dataset[i]
6         batch = {k: v.unsqueeze(0).to(device)
7                  for k, v in frame.items()
8                  if isinstance(v, torch.Tensor)}
9
10        action = policy.select_action(batch)
11        pred = action.squeeze().cpu().numpy()
12        gt = frame["action"].numpy()

```

select_action vs forward

- forward(): 학습 시 사용. loss를 반환
- select_action(): 추론 시 사용. 예측된 action을 반환
- reset(): 에피소드 시작 시 내부 상태(action queue 등) 초기화

실행 방법

```
python src/04_inference_no_robot.py
```

6 05_collect_data.py — 시연 데이터 수집

src/05_collect_data.py

Koch v1.1 로봇팔 필요

이 스크립트는 실제 Koch v1.1 Leader/Follower 로봇팔과 USB 웹캠이 연결되어 있어야 실행 가능하다.

6.1 목적

사람이 Leader 팔을 조종하여(텔레오퍼레이션) 작업을 시연하는 데이터를 녹화한다. 녹화된 데이터는 HuggingFace Hub에 업로드되어 SmolVLA Fine-tuning에 사용된다.

6.2 동작 흐름

1. **하드웨어 설정**: Follower 포트, Leader 포트, 카메라 인덱스 지정
2. **로봇 객체 생성**: KochFollower, KochLeader 인스턴스화
3. **데이터셋 생성**: LeRobotDataset.create()로 저장소 초기화
4. **녹화 루프**: 각 에피소드마다 record_loop() 호출
5. **Hub 업로드**: dataset.push_to_hub()

6.3 주요 코드 — 로봇 설정

```

1 robot_config = KochFollowerConfig(
2     port="/dev/ttyACM1",
3     id="my_koch_follower",
4     cameras={
5         "front": OpenCVCameraConfig(
6             index_or_path=0, width=640, height=480, fps=30,
7         )
8     },
9 )
10 robot = KochFollower(robot_config)
11 teleop = KochLeader(KochLeaderConfig(
12     port="/dev/ttyACM0", id="my_koch_leader",
13 ))

```

6.4 주요 코드 — 녹화 루프

```

1 for ep in range(NUM_EPISODES):
2     record_loop(
3         robot=robot, events=events, fps=FPS,
4         teleop=teleop, dataset=dataset,
5         control_time_s=EPISODE_LENGTH_SEC,
6         single_task="Pick up the block and place it in the bin",
7     )
8     dataset.save_episode()

```

사용 전 수정이 필요한 변수

- FOLLOWER_PORT, LEADER_PORT: lerobot-find-port로 확인
- HF_REPO_ID: 자신의 HuggingFace 사용자명으로 변경
- CAMERA_INDEX: 웹캠 인덱스 (0, 1, 2 등)

실행 방법

```
python src/05_collect_data.py
```

7 06_finetune_smolvla.py — SmolVLA Fine-tuning

```
src/06_finetune_smolvla.py
```

7.1 목적

lerobot/smolvla_base 사전학습 모델을 로드하고, 수집한 데이터셋(또는 공개 데이터셋)으로 Fine-tuning한다. 16GB 이상 VRAM GPU가 필요하다.

7.2 동작 흐름

1. SmolVLAPolicy.from_pretrained("lerobot/smolvla_base") 로드
2. make_pre_post_processors()로 전/후처리 파이프라인 생성
3. LeRobotDataset으로 데이터 로드, DataLoader 구성
4. AdamW 옵티마이저 설정 (lr=1e-4, weight_decay=1e-4)
5. 학습 루프: 전처리 → forward → loss → backward → step
6. 10,000 스텝마다 체크포인트 저장, 최종 모델 저장

7.3 주요 코드 — 학습 루프

```

1 policy = SmolVLAPolicy.from_pretrained("lerobot/smolvla_base")
2 policy.to(device).train()
3
4 preprocess, postprocess = make_pre_post_processors(policy.config, BASE_MODEL)
5 optimizer = torch.optim.AdamW(policy.parameters(), lr=1e-4)
6
7 while step < NUM_STEPS:
8     for batch in dataloader:
9         batch = preprocess(batch)
10        loss, outputs = policy.forward(batch)
11
12        optimizer.zero_grad()
13        loss.backward()
14        optimizer.step()
15
16        if step % 10000 == 0 and step > 0:
17            policy.save_pretrained(OUTPUT_DIR / f"checkpoint_{step}")

```

GPU 메모리 부족 시

- BATCH_SIZE를 4 → 2로 줄인다
- CLI 사용 시 --policy.gradient_checkpointing=true 추가
- --policy.train_expert_only=true: Action Expert만 학습 (VLM 동결)

실행 방법

```

# 를DATASET_ID 자신의 데이터셋으로 수정한 후 실행
python src/06_finetune_smolvla.py

```

8 07_run_policy_koch.py — 자율 제어

src/07_run_policy_koch.py

Koch v1.1 로봇팔 필요

이 스크립트는 실제 Koch v1.1 Follower 로봇팔과 웹캠이 필요하다. Leader 팔은 **연결하지 않는다** — 정책이 직접 행동을 생성한다.

8.1 목적

Fine-tuning된 SmolVLA 모델을 로드하여 Koch v1.1 로봇팔을 **자율 제어**한다. 각 에피소드 후 사용자가 성공 여부를 입력하여 성공률을 계산한다.

8.2 동작 흐름

1. Follower 로봇 + 카메라 설정 (Leader 없음)
2. SmolVLAPolicy.from_pretrained(MODEL_ID) 로드
3. 전처리/후처리 파이프라인 생성
4. 평가 데이터셋 생성 (로깅용)
5. 에피소드 루프: record_loop() 에 policy=policy 전달
6. 에피소드 종료 후 사용자에게 성공 여부 입력 받기 (y/n)
7. 전체 성공률 출력 및 Hub 업로드

8.3 주요 코드 — 자율 제어 실행

```

1 policy = SmolVLAPolicy.from_pretrained(MODEL_ID)
2 policy = policy.to(device).eval()
3
4 for ep in range(NUM_EVAL_EPISODES):
5     record_loop(
6         robot=robot, events=events, fps=30,
7         policy=policy,                    # 정책이 행동 생성
8         preprocessor=preprocessor,
9         postprocessor=postprocessor,
10        dataset=dataset,
11        single_task=TASK_DESC,
12    )
13    dataset.save_episode()
14
15    result = input(" 성공 여부 (y/n): ")
16    if result == "y":
17        successes += 1

```

05번(데이터 수집)과의 차이

- 05: teleop=teleop 전달 → 사람이 Leader로 조종
- 07: policy=policy 전달 → 모델이 자율적으로 행동 생성

- 나머지 구조(record_loop, 데이터셋 저장)는 동일

실행 방법

```
# MODEL_ID, FOLLOWER_PORT 등을 자신의 환경에 맞게 수정 후 실행  
python src/07_run_policy_koch.py
```

9 08_sim_smolvla_koch.py — SmolVLA + Koch 시뮬레이션

src/08_sim_smolvla_koch.py

9.1 목적

실물 로봇 없이 SmolVLA 모델을 로드하고, Matplotlib 기반 시뮬레이션 환경에서 Koch v1.1 로봇팔을 제어하는 **전체 파이프라인**을 시연한다. 3가지 모드를 제공한다.

9.2 모드 요약

Table 2: 08번 스크립트 실행 모드

모드	GPU 필요	설명
random_policy	아니오	SmolVLA 없이 랜덤 액션으로 시뮬레이션 파이프라인 검증
pretrained	예	lerobot/smolvla_base 사전학습 모델로 실시간 추론
collect_train	예	IK 전문가 데이터 수집 → fine-tune → 추론 시각화

9.3 내부 구성 요소

이 파일은 크게 3가지 클래스/모듈로 구성된다:

9.3.1 (1) Koch 시뮬레이션 엔진

Forward Kinematics (DH 파라미터)와 CCD 역기구학을 자체 구현하여, PyBullet 등 외부 물리 엔진 없이 Koch v1.1을 시뮬레이션한다.

```

1 def forward_kinematics(joint_angles):
2     """개5 관절 각도(rad) -> 개7 위치 (base ~ gripper tip)"""
3     # DH 파라미터 기반 연쇄 변환
4     # LINK_LENGTHS = [0.02, 0.045, 0.13, 0.124, 0.05, 0.045]
5     ...
6
7 def ik_ccd(target_pos, initial_angles=None, max_iter=300, tol=0.003):
8     """CCD(Cyclic Coordinate Descent) 역기구학"""
9     # 엔드 이펙터 -> 목표 위치까지 반복 수렴
10    ...

```

9.3.2 (2) KochSimEnv — 시뮬레이션 환경 클래스

SmolVLA와 연동하기 위한 환경 인터페이스를 제공한다.

```

1 class KochSimEnv:
2     def reset(self): # 환경 초기화
3     def get_state_6d(self): # [5 joints + gripper] 반환
4     def get_observation(self): # {state, image} 딕셔너리 반환
5     def apply_action(self, action_6d): # 절대 액션 적용
6     def apply_action_delta(self, delta_6d, scale=0.05): # 델타 액션 적용
7     def render_image(self, w, h): # RGB 이미지(numpy) 렌더링
8     def render_live(self, ax, info): # 실시간 3D 시각화

```

9.3.3 (3) SmolVLASimWrapper — SmolVLA 래퍼

SmolVLA 모델 로드, 전처리, 추론, 후처리를 하나의 인터페이스로 감싼다.

```

1 class SmolVLASimWrapper:
2     def __init__(self, device, pretrained_path):
3         # SmolVLAPolicy.from_pretrained() 로드
4         # 전처리후처리/ 파이프라인 설정
5         # 정규화 통계 로드 또는( 기본값 생성)
6
7     def predict_action(self, image_rgb, state_6d):
8         """
9         시물 카메라 이미지(224x224 uint8) + 관절 상태(6D)
10        -> SmolVLA 추론 -> 6D 액션 반환
11        """
12
13     def set_task(self, task_text):
14         """자연어 작업 명령 설정"""

```

9.4 SmolVLA 추론 파이프라인 상세

predict_action() 메서드의 내부 동작:

1. **이미지 전처리**: uint8 (H,W,3) → float32 [0,1] (3,H,W)
2. **배치 구성**: observation.state, observation.images.*, task 키로 딕셔너리 생성
3. **전처리 파이프라인**: 리사이즈, 정규화, 토큰화 등
4. **추론**: policy.select_action(processed) — 50-step action chunk 예측
5. **후처리**: 역정규화, CPU 이동
6. **반환**: 6D 액션 벡터 (5 관절 + 그리퍼)

```

1 def predict_action(self, image_rgb, state_6d):
2     img_tensor = torch.from_numpy(image_rgb).float().permute(2,0,1) / 255.0
3     state_tensor = torch.from_numpy(state_6d).float()
4
5     batch = {"observation.state": state_tensor, "task": self.task_text}
6     for key in image_keys:
7         batch[key] = img_tensor
8
9     processed = self.preprocessor(batch)
10    with torch.no_grad():
11        action = self.policy.select_action(processed)
12    result = self.postprocessor(action)
13    return result.squeeze().cpu().numpy()[:6]

```

9.5 모드별 실행

실행 방법

```

# 모드 1: 랜덤 정책 (GPU 불필요, 가장 빠른 테스트)
python src/08_sim_smolvla_koch.py

```



```
# 모드 2: 사전학습 모델 추론 (GPU 필요)
python src/08_sim_smolvla_koch.py --mode pretrained

# 모드 3: 수집 + 학습 + 추론 전체 파이프라인
python src/08_sim_smolvla_koch.py --mode collect_train --demos 30
```

Windows 호환

이 스크립트에는 Windows 11 호환 코드가 포함되어 있다:

- 콘솔 인코딩 설정: `sys.stdout.reconfigure(encoding="utf-8")`
- 한글 폰트: `plt.rcParams["font.family"] = "Malgun Gothic"`
- WSL2 또는 Windows 네이티브에서 모두 실행 가능

10 09_voice_pick_place.py — 음성 Pick & Place 시뮬레이션

src/09_voice_pick_place.py

10.1 목적

음성 또는 텍스트 명령으로 Koch v1.1 시뮬레이션 로봇팔에게 “물체를 잡아서 박스에 넣어”와 같은 작업을 지시한다. 실물 로봇, GPU 모두 불필요하며, **numpy + matplotlib**만으로 실행 가능하다.

08번과의 차이

- **08번**: SmolVLA 모델이 행동을 예측 (AI 추론 기반)
- **09번**: 자연어 명령을 파싱하여 IK 기반으로 실행 (규칙 기반), 음성 입력 지원
- 08번은 VLA 파이프라인 학습이 목적, 09번은 **음성 명령** → **로봇 동작** 시연이 목적

10.2 모드 요약

Table 3: 09번 스크립트 실행 모드

모드	추가 패키지	입력 방식	설명
text (기본)	없음	키보드	터미널에서 명령 입력
voice	SpeechRecognition, pyaudio	마이크	Google STT 음성 인식
auto	없음	자동	3개 명령 순차 자동 실행 (데모)

10.3 시뮬레이션 씬 구성

테이블 위에 3개 색상 블록과 1개 목표 박스가 배치된다.

Table 4: 시뮬레이션 오브젝트

오브젝트	색상	위치 (x, y)	역할
RED 블록	빨강	(0.12, 0.0)	잡기 대상
BLUE 블록	파랑	(-0.08, 0.05)	잡기 대상
GREEN 블록	초록	(0.0, -0.1)	잡기 대상
BOX	노랑	(0.2, 0.2)	목표 컨테이너

10.4 동작 흐름

1. **명령 입력**: 음성/텍스트로 자연어 명령 수신
2. **명령 파싱**: CommandParser가 색상/물체/컨테이너 키워드를 추출하여 (잡을 물체, 놓을 위치) 결정
3. **IK 경로 계획**: 잡기 위치, 놓기 위치에 대해 multi-start CCD 역기구학 계산
4. **Pick & Place 실행**: 접근 → 하강 → 잡기 → 상승 → 이동 → 하강 → 놓기 → 복귀
5. **3D 시각화**: 각 단계가 Matplotlib 3D 뷰에서 실시간 애니메이션

10.5 내부 구성 요소

10.5.1 (1) PickPlaceSimulator — 시뮬레이션 + 물리 엔진

08번의 KochSimEnv와 달리, 잡기/놓기 물리 판정이 포함되어 있다.

```
1 class PickPlaceSimulator:
2     def add_object(name, position, ...): # 오브젝트 배치
3     def try_pick(self): # 그리퍼 근처 물체 잡기 (XY 거리 < 8cm)
4     def try_place(self): # 잡은 물체 놓기
5     def update_picked(self): # 잡은 물체를 EE 위치로 추적
6     def execute_pick_and_place(self, pick_obj, place_obj, draw_callback):
7         # 단계9 시퀀스 자동 실행 (IK + 보간 + 물리)
8     def render(self, ax, status, command): # 3D 시각화
```

10.5.2 (2) CommandParser — 자연어 명령 파서

한국어와 영어 명령 모두 지원한다. 색상/물체/컨테이너 키워드를 매칭하여 (잡을 물체, 놓을 위치)를 결정한다.

```
1 class CommandParser:
2     # 색상: 빨간", 파랑", "red", "blue", ...
3     # 물체: 블록", 물체", "block", "object", ...
4     # 컨테이너: 박스", 상자", "box", "bin", ...
5
6     def parse(self, text, sim):
7         """'빨간' 블록을 박스에 넣어' -> (red_block, box)"""
8         # 1. 색상 키워드 -> pick 대상 검색
9         # 2. 컨테이너 키워드 -> place 대상 검색
10        # 3. 키워드 없으면 기본 물체컨테이너/ 선택
11        return (pick_object, place_object)
```

인식 가능한 명령 예시:

- "빨간 블록을 박스에 넣어" — 한국어 (색상 + 컨테이너)
- "파란 물체를 상자에 옮겨" — 한국어 (색상 + 동작)
- "pick up the green block and put it in the box" — 영어
- "초록 블록 박스" — 최소 키워드 (색상 + 컨테이너만으로 충분)

10.5.3 (3) VoiceInput — 음성 인식

Google Speech-to-Text API를 사용하며, 마이크 또는 패키지가 없으면 자동으로 키보드 입력으로 전환된다.

```
1 class VoiceInput:
2     def __init__(self, lang="ko-KR"):
3         import speech_recognition as sr
4         self.recognizer = sr.Recognizer()
5         self.microphone = sr.Microphone()
6         # 주변 소음 자동 보정
7
8     def listen(self, timeout=5):
9         """마이크 -> 텍스트 (Google STT)"""
10        audio = self.recognizer.listen(source, timeout=timeout)
11        return self.recognizer.recognize_google(audio, language=self.lang)
```

10.5.4 (4) Multi-Start CCD 역기구학

단일 초기값 CCD는 로봇 자세에 따라 수렴 실패할 수 있다. 여러 초기 자세 후보 중 오차가 가장 작은 결과를 선택한다.

```

1 def ik_ccd(target_pos, initial_angles=None, max_iter=600, tol=0.002):
2     """Multi-start CCD: 여러 초기 자세 후보 중 최적 선택"""
3     base_angle = math.atan2(target[1], target[0])
4     candidates = [
5         [base_angle, 0.5, -0.5, -0.3, 0.0], # 앞으로 뻗은 자세
6         [base_angle, 0.8, -0.8, -0.2, 0.0], # 더 앞으로
7         [base_angle, 0.3, -0.3, -0.5, 0.0], # 아래로
8         ...
9     ]
10    # 각 후보에서 IK 풀이 -> 오차 최소인 결과 반환

```

10.6 Pick & Place 시퀀스 상세

execute_pick_and_place() 메서드는 9단계로 실행된다:

1. 접근: 잡기 위치 상공 (z + 8cm)으로 이동
2. 하강: 잡기 위치로 하강
3. 잡기: 그리퍼 닫기 + try_pick() 물리 판정
4. 상승: 잡은 채로 상공 복귀
5. 중간점: pick과 place 중간 높은 위치 경유 (IK 안정성)
6. 목표 접근: 놓기 위치 상공으로 이동
7. 하강: 놓기 위치로 하강
8. 놓기: 그리퍼 열기 + try_place() 물리 판정
9. 복귀: 상공으로 복귀

각 단계는 50스텝으로 보간되며, 매 스텝마다 3D 시각화가 업데이트된다.

10.7 실행 방법

실행 방법

```

# === 텍스트 입력 모드 기본(, 추가 패키지 불필요) ===
python src/09_voice_pick_place.py

# === 자동 데모 개(3 블록을 순서대로 박스에 넣음) ===
python src/09_voice_pick_place.py --mode auto

# === 음성 인식 모드 추가( 패키지 필요) ===
pip install SpeechRecognition pyaudio
python src/09_voice_pick_place.py --mode voice

# === 영어 음성 인식 ===
python src/09_voice_pick_place.py --mode voice --lang en-US

```

텍스트 모드 사용법

실행하면 3D 시각화 창이 열리고, 터미널에서 명령을 입력한다:

```
명령
> 빨간 블록을 박스에 넣어
  -> 잡기: RED | 놓기: BOX
  -> 성공! 거리(: 1.8cm)명령

> blue block box
  -> 잡기: BLUE | 놓기: BOX
  -> 성공! 거리(: 2.4cm)명령

> quit
```

음성 모드 패키지 설치

voice 모드는 추가 패키지가 필요하다:

```
pip install SpeechRecognition pyaudio
```

Windows에서 pyaudio 설치가 실패하면:

```
pip install pipwin
pipwin install pyaudio
```

패키지 없이도 text 모드와 auto 모드는 정상 동작한다.

11 스크립트 간 관계 요약

Table 5: 스크립트 간 입출력 관계

스크립트	입력 (의존)	출력 (산출물)
01	(없음)	설치 상태 확인
02	Hub 데이터셋	데이터 구조 이해, .png
03	Hub 데이터셋 + GPU	outputs/train/diffusion_pusht/ 체크포인트
04	Hub 사전학습 모델	예측 vs GT 비교 출력
05	Koch 로봇 + 카메라	Hub 데이터셋 (koch_pick_place)
06	05의 데이터셋 + GPU	outputs/train/smolvla_finetuned/ 체크포인트
07	06의 모델 + Koch 로봇	성공률, Hub 평가 데이터
08	GPU (optional)	시뮬레이션 시각화, 학습 데이터
09	(없음)	음성/텍스트 명령 → 3D Pick & Place 시각화

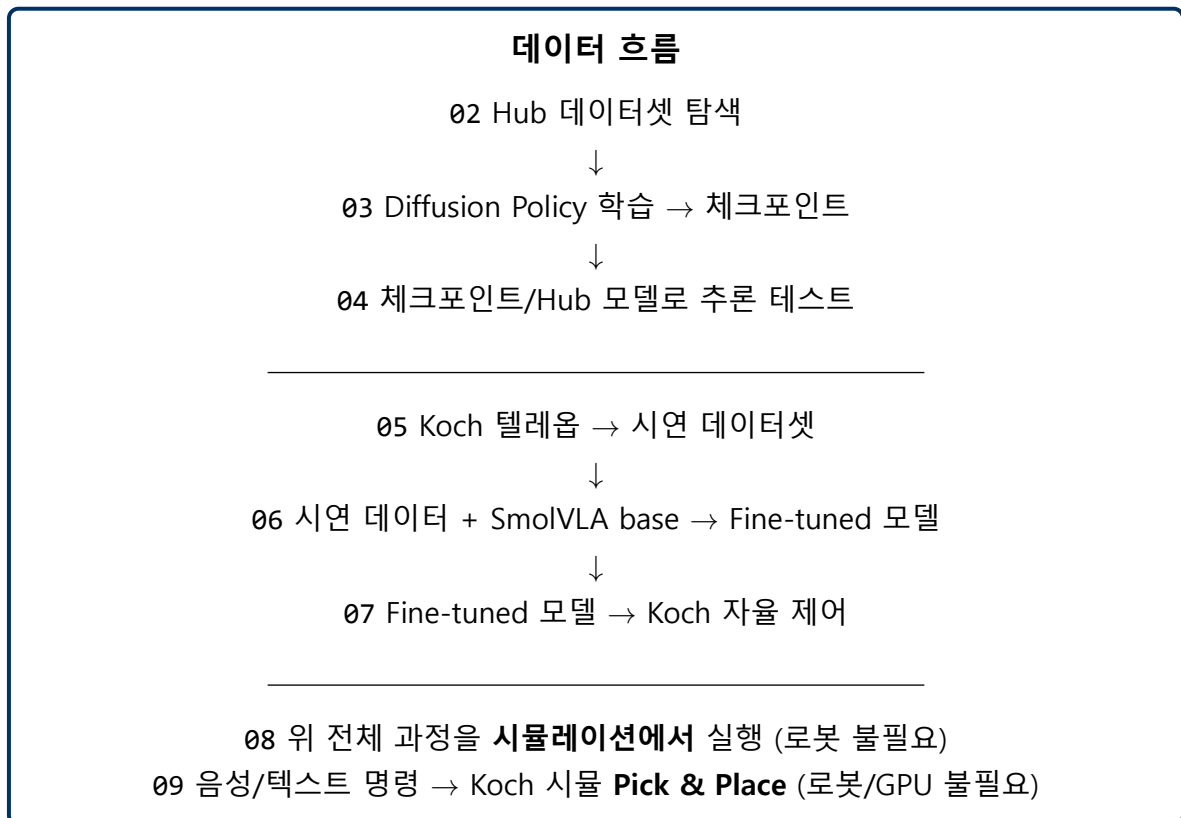


Figure 2: 스크립트 간 데이터 흐름